Software Requirements Specification

Smart Campus Study Room Reservation and Seat Management System (StudySeat)

Version 2.0 · 16 October 2026 · StudySeat Team (Zhi Xing You Zuo)

Item	Content
Project name	Smart Campus Study Room Reservation and Seat Management System
Product name	StudySeat
Team	StudySeat Team (Zhi Xing You Zuo), 13 members; team leader Sun Jun, presenter Zeng Jingkang
Course	Software Engineering Practice, autumn term 2026; course community 2601_FZU-MU_SE
Document version	V2.0 (requirement analysis submission)
Date	16 October 2026
Prepared by	Shen Ziheng (requirements and rule review), Shi Qing (requirements and acceptance criteria), Sun Jun (coordination)
Reviewed by	Sun Jun (team leader), Wu Yiming (editor)
Intended readers	Development team members, teaching assistants, the instructor, and students who maintain the system later
Related material	Requirements analysis presentation and team blog; links are listed in Appendix B

Document revision history

Table 0-1 Document revision history

Version	Date	Changes	Author
V1.0	2026-10-09	Submitted with Assignment 2: first requirements specification for the team formation and topic selection stage, covering purpose and scope, users and environment, business rules and states, use cases, data and non-functional requirements, acceptance tests and open items	Shi Qing, Shen Ziheng
V2.0	2026-10-16	This submission: added user scenarios and traceability (Chapter 3), the class diagram and class descriptions, interface prototypes, numbered functional requirements, interfaces and error codes, interface and functional acceptance criteria, and a requirements traceability matrix; expanded concurrency and idempotency requirements	Shen Ziheng, Shi Qing, Sun Jun

Contents

1 Introduction 5

1.1 Purpose 5

1.2 Project background 5

1.3 Terms, definitions and abbreviations 6

1.4 References 7

1.5 Changes in this version 7

2 Overall description 8

2.1 Product objectives 8

2.1.1 Development intention 8

2.1.2 Application goals and scope 8

2.1.3 Product prospects 9

2.2 User characteristics and operating environment 9

2.3 Requirement elicitation and validation 10

2.4 Design constraints and assumptions 10

2.5 Requirement priorities and scope 11

3 User scenarios 12

3.1 Roles and permissions 12

3.2 Scenario overview 12

3.3 Scenario descriptions 13
US-01 Search available seats 13
US-02 Submit a booking 14
US-03 Two students book the same seat 14
US-04 Check in 15
US-05 Temporary leave and return 15
US-06 End the session 15
US-07 Release after a missed check-in 16
US-08 Cancel a booking 16
US-09 Configure opening hours and rules 17
US-10 Close a resource temporarily 17
US-11 Query records and handle a dispute 18
US-12 View personal bookings and attendance 18
3.4 Scenarios and functional requirements 18
4 Specific requirements 19
4.1 Functional requirements 19
4.2 Business rules and state model 20
4.3 Class diagram 24
4.4 Data and consistency requirements 27
4.5 Interface requirements 28
4.6 Non-functional requirements 29
5 Interface prototype 31
5.1 Design principles and information architecture 31
5.2 Student interface 31
5.3 Administrator interface 34
5.4 Screen sequence for the main flow 34
5.5 Interface consistency and usability checks 35
6 Functional description and acceptance criteria 36
6.1 Detailed functional description 36
6.1.1 Sign-in and permissions (FR-01, FR-02) 36
6.1.2 Seat search and resource states (FR-03, FR-12) 36

6.1.3 Booking submission and conflict check (FR-04, FR-05) 37
6.1.4 Check-in, temporary leave and return (FR-06, FR-07) 37
6.1.5 Session end and timeout release (FR-08, FR-09) 37
6.1.6 Administrator configuration, closures and record queries (FR-10 to FR-15) 38
6.2 Input and output formats 38
6.3 Interface acceptance criteria 39
6.4 Functional acceptance criteria 40
6.5 Requirements traceability 41
7 Open items and change management 43
7.1 Open items 43
7.2 Change management 43
7.3 Risks and responses 44
Appendix A Interview guide 45
A.1 Questions for students 45
A.2 Questions for administrators 45
Appendix B Deliverables and links 45

1 Introduction

1.1 Purpose

This document defines the functions, business rules, interfaces and acceptance criteria that the Smart Campus Study Room Reservation and Seat Management System (StudySeat) has to deliver at this stage. It serves three kinds of readers: the development team, who use it to agree on scope and rule definitions; the course teaching assistants and instructor, who use it to judge whether the requirements are clear, complete and verifiable; and the students who maintain the system later, who use it to understand why the system is designed this way. Implementation details are left out, and interface fields appear only where the interaction contract needs to be stated.

This is the requirement analysis deliverable and it replaces the first requirements specification (V1.0) produced earlier. Anything not yet confirmed by interviews or testing is marked as an open item and collected in Chapter 7, so that assumptions are not presented as conclusions.

1.2 Project background

Table 1-1 Project information

Item	Content
Product name	Smart Campus Study Room Reservation and Seat Management System (StudySeat)
Context	Software Engineering Practice, requirement analysis stage
Developers	StudySeat Team, 13 members, led by Sun Jun
Intended users	Students who book and use seats; study room administrators who maintain resources and rules
Earlier work	Assignment 1 delivered a front-end/back-end separated calculator; Assignment 2 produced the team plan and version V1.0 of this document

The topic comes from study routines around us. What we observed while choosing the topic is that students often need to judge in advance whether a seat will be free at a given time; there is no shared understanding of what happens when a booking is not used or when someone steps away briefly; and when a dispute arises, no record exists to check. How often these problems actually occur on campus has not yet been confirmed through formal interviews; Section 2.3 states how we intend to confirm it and where we currently stand.

One point needs saying early: the university may already run a reservation platform. The first thing the project does is therefore not coding but finding out what that platform already covers. If it satisfies the main needs, the team will narrow the scope as described in Section 2.1.3 and concentrate on what is still missing.

1.3 Terms, definitions and abbreviations

The table below defines the terms used in this document. The same wording is used when a term first appears in the text, in interface labels and in data fields, so that one concept is not described in several different ways.

Table 1-2 Terms, definitions and abbreviations

Term or abbreviation	Full form	Meaning
SRS	Software Requirements Specification	This document
StudySeat	—	Product name and team name
User	User	An account that signs in and holds a role
Student	Student	The role that books seats, checks in and ends sessions
Administrator	Administrator	The role that maintains rooms, seats, opening hours and rules
Booking / reservation	Reservation	A student request to occupy one seat for one time interval
Slot	Slot	A half-open time interval written as [start, end)
Check-in	Check-in	The student confirms attendance inside the check-in window; the booking moves from pending to in use
Temporary leave	Temporary leave	Leaving the seat briefly during a session and returning to it
Expired	Expired	No check-in inside the window, or a temporary leave longer than the limit; the system ends the booking
Release	Release	Making the seat bookable again in the affected period after a cancellation or timeout
Idempotent	Idempotent	Repeating the same request does not change a recorded result or cause extra side effects
Transaction	Transaction	A group of database operations that either all commit or all roll back
Conflict	Conflict	Two valid bookings for the same seat or the same student in overlapping intervals
API	Application Programming Interface	The contract used by the front end to call the back end; data is exchanged as JSON

Term or abbreviation	Full form	Meaning
RBAC	Role-Based Access Control	Access control based on user roles
TBD	To Be Determined	Marker for open items, collected in Chapter 7

1.4 References

Table 1-3 References

No.	Reference	Use
R1	Course requirements for requirement analysis (bbs.csdn.net/topics/620549144)	Structure and marking criteria of this document
R2	Team blog for Assignment 2 and team CSDN home page, https://blog.csdn.net/2501_91679864	Source of the topic argument and of V1.0; also the submission entry for course work
R3	GB/T 8567-2006, Specification for computer software documentation	Document structure and writing rules
R4	ISO/IEC/IEEE 29148:2018, Systems and software engineering — Life cycle processes — Requirements engineering	Requirement verifiability

1.5 Changes in this version

This stage asks for a further improvement of the previous requirements specification. V1.0 already set out the business rules, states and use cases clearly; what it lacked was scenario level description, a class diagram, interface prototypes and acceptance criteria that can be checked one by one. This version rewrites those four parts and adds a requirement numbering scheme so that requirements, scenarios and acceptance cases can be traced to one another.

Table 1-4 Changes from V1.0 to V2.0

Area	State in V1.0	Handling in V2.0
User scenarios	Only use cases UC01 to UC03	Extended to 12 user scenarios covering search, booking, concurrency, check-in, temporary leave, session end, timeout, cancellation, configuration, resource closure, record queries and personal records (Chapter 3)
Class diagram	Not provided	Domain class diagram with class descriptions and associations, including the reasoning for placing concurrency control in the service layer (Section 4.3)
Interface prototype	Not provided	Interface concept image, student and administrator wireframes, and interface acceptance criteria (Chapter 5 and Section 6.3)

Area	State in V1.0	Handling in V2.0
Functional requirements	Described in prose without identifiers	Numbered requirements FR-01 to FR-15 with inputs, processing, outputs and priority (Sections 4.1 and 6.1)
Interfaces and error codes	Not defined	Main interfaces and error codes, with the page behaviour expected on failure (Section 4.5)
Acceptance criteria	Textual statements only	Interface acceptance criteria and 15 functional acceptance cases, with a traceability matrix (Sections 6.3, 6.4 and 6.5)
Consistency requirements	Stated as principles	Uniqueness constraints, transaction boundaries, concurrency control and idempotency requirements (Sections 4.2 and 4.4)

2 Overall description

2.1 Product objectives

2.1.1 Development intention

The course asks us to walk through requirement analysis, design, implementation and testing in one project. A study room booking system was chosen because its process boundary is clear and because it naturally contains concurrent resource allocation, state transitions and configurable rules, which makes it a good exercise in stating the rules before writing code.

The team does not want a demonstration that only shows screens. The test we set ourselves is simple: can the rules be explained by the system? Why a seat allows only one valid booking, how a timeout is decided, and whether a record can be found when a student disputes an outcome all have to be answered during requirement analysis, not patched in later during testing.

2.1.2 Application goals and scope

Version 1 covers study rooms on a single campus and includes the main flows for students and administrators. Students can sign in, search seats by date and time, create and cancel bookings, check in, take a temporary leave and return, end a session, and review their own bookings and attendance records. Administrators can maintain rooms and seats, configure opening hours and rule parameters, close a resource temporarily, and query or

export records. The system is used through a browser on desktop and mobile; version 1 works with test accounts and simulated study room data.

The following are explicitly out of scope: payment, face recognition, integration with physical access gates, booking across campuses, recommendations based on past behaviour, and SMS or WeChat notifications. These features depend on authorisation, hardware or additional data that cannot be verified within the course period; including them would blur the scope.

2.1.3 Product prospects

If the course demonstration and feedback from classmates show that the rules work, the next step is a pilot in one study room. A pilot needs at least three things: authorisation from the venue administrator, a named person responsible for maintaining the system, and a channel for collecting user feedback. Until those exist, the system stays at demonstration and test level and no real student data is used.

Rule parameters live in configuration (Section 4.2), so a pilot can change the check-in window or the temporary leave limit without touching code: the administrator confirms the new values and saves them as a new version. This is the main difference between this system and a one-off demonstration.

2.2 User characteristics and operating environment

Table 2-1 User roles and usage characteristics

Role	Typical user	Main terminal	Frequency	Characteristics and expectations
Student	Student who needs a regular place to study	Mobile browser mainly, desktop secondarily	Several times a week	Familiar with mobile apps and expects to finish a booking in three steps; sensitive to failure messages and wants to know why a seat could not be booked and what to do next
Administrator	Study room staff member	Desktop browser	Once or twice a week for configuration and queries	Familiar with spreadsheet tools; cares about rules that can be explained and records that can be traced, and expects to change parameters without help from a developer

The back end is implemented in Python, interfaces exchange JSON, and data is stored in a relational database. The client is a browser page that adapts to common desktop and mobile resolutions. Time is taken from the server and displayed in Beijing time;

decisions about the check-in window and timeouts are made on the server, never from the client clock, so that changing a device clock cannot bypass the rules.

2.3 Requirement elicitation and validation

Requirements come from three sources: the daily situations students face, the management requirements of the administrators, and the capability boundary of the existing platform. To avoid treating our own impressions as user needs, the table below lists how each requirement will be elicited and validated; the parts not yet done are marked as open items and given an owner and time in Chapter 7.

Table 2-2 Requirement elicitation and validation methods

Method	Participants and size	Purpose	Current status
Semi-structured interviews	At least 10 students	Confirm how often finding a seat, occupying a seat and stepping away are actual problems, and how they are handled today	Planned, not yet conducted (TBD-01)
Administrator interview	At least one study room administrator	Confirm opening hours, check-in and temporary leave rules in practice, and how exceptions and disputes are handled	Planned, not yet conducted (TBD-02)
Survey of existing platforms	Reservation or seat management tools already used on campus	Identify which needs are already met so that the project does not duplicate them	Planned (TBD-03)
Usability test	Several classmates	Check whether a booking task can be completed without guidance and observe how users recover from failures	After development (related to AC-01 to AC-04)
Rule and concurrency testing	100 concurrent virtual users	Verify that conflicting bookings cannot both succeed and that cancellations and timeouts release resources correctly	After development (AC-05, AC-06, AC-15)

One point needs to be explicit: values such as four hours, seven days, the check-in window and a 30-minute leave in this document come from the initial proposal in Assignment 2. They are provisional, not current university regulations. They must be confirmed by an administrator before they take effect in the system (TBD-02, TBD-05).

2.4 Design constraints and assumptions

- **Course period:** Main development is concentrated in weeks 3 to 6, so the scope follows the priority order in Section 2.5 and covers the must-have items first.

- **Venue and data:** There is no venue authorisation and no operational data, so the demonstration uses simulated study rooms and test accounts and never real student data.
- **Authentication:** Until campus single sign-on is authorised, the system uses its own accounts; integration is an open item before any pilot (TBD-07).
- **Assumption 1:** Opening hours are fairly stable and can be maintained weekly by an administrator, with temporary changes handled by closing a resource.
- **Assumption 2:** Students mostly use a mobile browser on a network that is available but not always fast; the pages still have to explain failures and offer a retry.
- **Assumption 3:** With the application server and database on the same network segment, main interfaces respond within one second, leaving room for the performance target of two seconds for 95 percent of requests at 100 concurrent users.

If these assumptions do not hold, for example if the university platform already covers seat level booking, the scope is adjusted as described in Section 2.1.3 and the reason is recorded in the blog post.

2.5 Requirement priorities and scope

Priorities use four levels: must have, should have, could have and will not have. The ordering follows two questions: does the core flow still work without it, and can the rules still be verified. Anything without which the booking and release flow cannot be demonstrated is a must-have.

Table 2-3 Requirement priorities and scope for this version

Priority	Basis	Requirements included
Must have	Without it the core flow and rule verification cannot be demonstrated	FR-01 sign-in and role detection, FR-03 seat search, FR-04 booking submission, FR-05 conflict check, FR-06 check-in, FR-07 temporary leave and return, FR-08 session end and timeout release, FR-12 rule configuration
Should have	Affects how explainable the rules are, how traceable the records are, and the cost of maintenance	FR-02 account and password management, FR-09 cancellation, FR-10 resource closure, FR-11 record query, FR-13 operation log
Could have	Added when time allows; not needed for acceptance of the core flow	FR-14 in-system notifications and reminders, FR-15 data export
Will not have	Outside the course scope or dependent on external authorisation	Payment, face recognition, access gate integration, cross-campus booking, recommendations, SMS and WeChat notifications

3 User scenarios

This chapter describes the situations in which the system has to help, from the point of view of the people using it. Each scenario states the trigger, the main flow, the exceptions and the state the system is left in, and together they form the basis for the functional requirements and acceptance cases that follow. Rule identifiers refer to the business rules in Section 4.2.

3.1 Roles and permissions

Table 3-1 Roles and permissions

Role	Main goal	Allowed actions	Visible data
Student	Find and book a free seat, check in on time and end the session	Search seats, create and cancel bookings, check in, take a temporary leave and return, end the session, view personal records	Own bookings and attendance records, plus public information about seats and rooms
Administrator	Maintain resources and rules, handle exceptions and disputes	Maintain rooms, seats, opening hours and rule parameters; close resources; query all records; handle abnormal bookings	All bookings, attendance records and operation logs, excluding passwords
System (scheduled task)	Keep booking states and seat occupancy consistent	Handle bookings with no check-in or an expired leave, release the corresponding periods, write operation logs	Booking and seat states

Permissions are enforced on the server and do not rely on hiding buttons in the browser. A student who requests someone else's booking number receives a permission error rather than data, and administrator interfaces reject student accounts with 403 (see AC-14).

3.2 Scenario overview

Table 3-2 Scenario overview

ID	Scenario	Main role	Trigger	Requirements
US-01	Search available seats	Student	Date and time chosen on the search page	FR-03
US-02	Submit a booking	Student	A free seat is confirmed	FR-04, FR-05
US-03	Two students book the same seat	Two students	Both submit overlapping bookings for one seat	FR-04, FR-05
US-04	Check in	Student	Booking is pending and the window is open	FR-06

ID	Scenario	Main role	Trigger	Requirements
US-05	Temporary leave and return	Student	The student steps away during a session	FR-07
US-06	End the session	Student	Study finishes early	FR-08
US-07	Release after a missed check-in	System	The check-in window closes	FR-08
US-08	Cancel a booking	Student	Plans change and the student cannot attend	FR-09
US-09	Configure opening hours and rules	Administrator	Term start or a rule adjustment	FR-12
US-10	Close a resource temporarily	Administrator	Maintenance or an event	FR-10
US-11	Query records and handle a dispute	Administrator	A student reports a problem	FR-11, FR-13
US-12	View personal bookings and attendance	Student	The student checks the booking state	FR-11

3.3 Scenario descriptions

US-01 Search available seats

Role and preconditions: The student is signed in; the administrator has configured rooms, seats and opening hours.

Trigger: The student selects 2026-10-18 and the period 14:00-16:00 on the search page.

Main flow:

- The student chooses a date, a time period and optional filters such as capacity or seat type.
- The system calculates occupancy for the chosen period and returns the seats that are free.
- The list shows the study room, seat number, seat type and current state.
- The student narrows the filters again or selects a seat and moves to the confirmation page.

Exceptions: If the period is outside opening hours the system reports that the period is not open and returns no seats; if the filters are too narrow the page shows an empty state with suggestions for nearby periods; if the request fails the page offers a retry and keeps the chosen filters.

Resulting state: The student sees seats that match the filters; nothing is written to the database.

Related rules: BR-03, BR-05, BR-07.

US-02 Submit a booking

Role and preconditions: A free seat and time period have been selected and the student holds no other valid booking in that period.

Trigger: The student selects “Confirm booking” on the confirmation page.

Main flow:

- The page shows the seat, date and period together with the rule reminders (maximum length, lead time, check-in window).
- The server checks the rules: length against the maximum, lead time against seven days, the period against opening hours, and whether the student already holds a valid booking in that period.
- In the same transaction it re-checks seat occupancy for the period, writes the booking and generates a booking number.
- The booking number and the check-in window are returned and the page shows the check-in entry.

Exceptions: If the seat was taken a moment earlier the system reports “seat already taken”, keeps the chosen filters and lets the student pick another seat; if the student already holds a booking in that period the conflict and the existing booking number are shown; if the length or lead time is exceeded the corresponding limit is reported; repeated submissions are handled idempotently and do not create a second booking.

Resulting state: A booking exists in the pending state, the seat counts as occupied for that period, and the key action is logged.

Related rules: BR-01 to BR-06, BR-09, BR-10.

US-03 Two students book the same seat

Role and preconditions: Students A and B both see the same seat as free for the same period, which is normal under concurrency.

Trigger: Both submit a booking for the same seat and overlapping periods within a very short time.

Main flow:

- The two requests reach the server almost simultaneously.
- The server checks occupancy and attempts the write inside one transaction; the database guarantees at most one valid booking per seat and period.
- The order in which the transactions commit decides the outcome: only one succeeds.
- The successful student receives a booking number; the other receives a clear reason, “seat already taken”.
- The unsuccessful student searches again and picks a different seat.

Exceptions: If a deadlock or transaction timeout causes a rollback, the request returns a retryable error and no partial write is left behind, so a seat is never marked occupied without a valid booking.

Resulting state: At most one valid booking exists for that seat and period.

Related rules: BR-05, BR-10. Acceptance case AC-05.

US-04 Check in

Role and preconditions: A booking is pending and the current time is inside the check-in window, which runs from 10 minutes before to 15 minutes after the start and includes the later boundary.

Trigger: The student selects check-in in “My bookings”. Check-in is a page action by default; whether scanning or on-site confirmation is added is an open item (TBD-04).

Main flow:

- The system checks the booking state and the server time.
- The state changes from pending to in use and the check-in time and method are recorded.
- The page shows the remaining session time together with the leave and end-session actions.

Exceptions: Before the window opens the system states that check-in is not open yet and shows the opening time; after the window closes the booking has already been marked expired and the page reports that the seat has been released; repeated check-ins keep the first recorded time and do not write another attendance record.

Resulting state: The booking is in use and one check-in record has been added.

Related rules: BR-04, BR-09, BR-12.

US-05 Temporary leave and return

Role and preconditions: The booking is in use and the leave limit has been confirmed by an administrator.

Trigger: The student needs to step away briefly, for example to collect a parcel.

Main flow:

- The student selects “Leave”; the system records the start of the leave and the state becomes on leave.
- The page shows the remaining time to return and reminds the student as the limit approaches.
- The student selects “Return” within 30 minutes; the state returns to in use and the leave time counts towards the session.

Exceptions: If the student does not return within 30 minutes the system marks the booking expired, ends it and releases the seat; ending the session during a leave is allowed; repeated leave requests are idempotent, and whether a new request restarts the timer is an open item (TBD-05).

Resulting state: The booking is back in use with the attendance timeline retained, or it has expired and the seat is released.

Related rules: BR-08, BR-09.

US-06 End the session

Role and preconditions: The booking is in use or on a temporary leave.

Trigger: The student finishes studying and selects “End session”.

Main flow:

- The system checks that the current state allows the action.
- The state becomes completed and the end time is recorded.
- The remaining part of the booked period becomes bookable again.
- The page shows the session length and the end time.

Exceptions: Repeated end requests are idempotent and do not change the end time; if the booking has already expired the expired state is kept and the seat is not released twice.

Resulting state: The booking is terminal and later periods can be booked by other students.

Related rules: BR-06, BR-09.

US-07 Release after a missed check-in

Role and preconditions: A pending booking has passed its check-in window without a check-in.

Trigger: A scheduled task scans for expired bookings at a fixed interval; a query by a student or administrator also triggers a check.

Main flow:

- The system finds pending bookings whose check-in window has closed.
- The state becomes expired and the decision time is recorded.
- The seat becomes bookable again for the remaining period.
- An operation log entry is written and the student sees the expired state and its reason in “My bookings”.

Exceptions: If the scheduled task is interrupted or delayed, it catches up after recovery; read and write interfaces also check for expiry so that an invalid booking cannot block a seat for long.

Resulting state: The expired booking no longer occupies the resource, and the record remains for later reference.

Related rules: BR-04, BR-06, BR-12. Acceptance case AC-06.

US-08 Cancel a booking

Role and preconditions: The booking is pending.

Trigger: The student cannot attend and cancels in advance.

Main flow:

- The student selects cancel and the system warns that the seat will be released immediately.
- After confirmation the state becomes cancelled and the time and operator are recorded.
- The seat becomes bookable again for the same period.

Exceptions: A booking that has been checked in, completed or expired cannot be cancelled; the page points to ending the session or another handling path instead. Repeated cancellations are idempotent.

Resulting state: The booking is terminal and the history is kept for queries.

Related rules: BR-06, BR-09.

US-09 Configure opening hours and rules

Role and preconditions: An administrator is signed in and needs to set up or adjust resources and rules.

Trigger: Setting up rooms at the start of term, or adjusting parameters in practice.

Main flow:

- The administrator maintains the list of study rooms and seats, including seat type and availability.
- Weekly opening hours are configured, and exceptions can be added for specific dates.
- Rule parameters are entered: maximum booking length, lead time, check-in window and maximum leave.
- Saving validates that the values are positive and within the allowed ranges.
- The system creates a new rule version and writes an operation log entry.

Exceptions: Unreasonable values, such as a negative check-in window, are rejected and the field is highlighted; changes that would affect existing bookings apply only to new bookings, while bookings already under way keep the previous parameters.

Resulting state: The new rules are in effect and each version is recorded for later review.

Related rules: BR-01 to BR-04, BR-11. Acceptance case AC-10.

US-10 Close a resource temporarily

Role and preconditions: An administrator is signed in and a room or seat must be taken out of service for a period.

Trigger: Room maintenance, an examination or a temporary event.

Main flow:

- The administrator selects the study room or seat, the closure period and the reason.
- The system lists the affected bookings with booking number, student and period.
- After confirmation the affected bookings are cancelled and an audit record is written.
- An in-system notice explains the reason and suggests an alternative study room.
- No new bookings are accepted for the closed period.

Exceptions: If an affected booking is already in use the page asks for on-site handling and lets the student end the session; a mistaken closure is corrected by reopening the period, since the closing action itself cannot be undone but is recorded.

Resulting state: No valid booking remains inside the closed period and the handling is traceable.

Related rules: BR-07, BR-11, BR-13. Acceptance case AC-11.

US-11 Query records and handle a dispute

Role and preconditions: An administrator is signed in; a student reports a seat problem or questions an expiry decision.

Trigger: Student feedback or a routine review of resource usage.

Main flow:

- The administrator queries bookings by date, seat or student number.
- The state change times, check-in and leave records and the related operation log are reviewed.
- Responsibility is judged from the records, for example whether check-in happened inside the window or the leave limit was exceeded.
- The case is handled according to the rules, explained to the student, and the conclusion is recorded in the log.

Exceptions: If records are missing or times are inconsistent, the retention period (TBD-06) is explained and the case is escalated to the maintainer instead of being judged from memory.

Resulting state: The dispute is handled with evidence and the conclusion is recorded.

Related rules: BR-13.

US-12 View personal bookings and attendance

Role and preconditions: The student is signed in and has current or past bookings.

Trigger: The student wants to confirm a booking state or review the attendance timeline.

Main flow:

- The student opens “My bookings” and the system lists bookings in reverse time order.
- The student reviews check-in, leave, return and end times for one booking.
- For states that allow actions, the page offers cancel, check-in, leave, return or end.

Exceptions: If there are no records the page shows an empty state and a link to seat search.

Resulting state: The student knows the booking state and which actions are possible next.

Related rules: BR-09.

3.4 Scenarios and functional requirements

Table 3-3 Mapping from scenarios to functional requirements

Scenario	Functional requirements	Business rules
US-01 Search available seats	FR-03	BR-03, BR-05, BR-07
US-02 Submit a booking	FR-04, FR-05, FR-13	BR-01 to BR-06, BR-09, BR-10
US-03 Concurrent booking of one seat	FR-04, FR-05	BR-05, BR-10

Scenario	Functional requirements	Business rules
US-04 Check in	FR-06, FR-11	BR-04, BR-09, BR-12
US-05 Temporary leave and return	FR-07	BR-08, BR-09
US-06 End the session	FR-08	BR-06, BR-09
US-07 Release after a missed check-in	FR-08, FR-13	BR-04, BR-06, BR-12
US-08 Cancel a booking	FR-09	BR-06, BR-09
US-09 Configure opening hours and rules	FR-12, FR-13	BR-01 to BR-04, BR-11
US-10 Close a resource temporarily	FR-10, FR-13, FR-14	BR-07, BR-11, BR-13
US-11 Query records and handle a dispute	FR-11, FR-13	BR-13
US-12 View personal bookings and attendance	FR-11	BR-09

4 Specific requirements

4.1 Functional requirements

The table below numbers everything the system has to do and serves as the shared basis for design, implementation and acceptance. Priorities follow Section 2.5, and the source scenario column points to the situations described in Chapter 3. How each function is processed, including failure handling, is expanded in Section 6.1, and how it is verified in Section 6.4.

Table 4-1 Functional requirements

ID	Function	Description	Main input	Main output	Priority	Scenario
FR-01	Sign-in and role detection	Sign in with an account and password; the server identifies the role and enables the matching functions	Username, password	Sign-in result and role	Must	US-01
FR-02	Account and password management	Administrators maintain test accounts; users may change their password, stored as a secure hash	Account data, new password	Result of the operation	Should	US-12
FR-03	Seat search	Return seats that are free for a date and period, with filters and seat states	Date, period, filters	List of available seats	Must	US-01

ID	Function	Description	Main input	Main output	Priority	Scenario
FR-04	Booking submission	Validate the rules, create the booking and return the booking number and check-in window	Seat, period, student	Booking number or failure reason	Must	US-02
FR-05	Conflict check	Check seat and student conflicts inside one transaction so that no seat or student overlaps	Seat, period, student	Allow or reject with a reason	Must	US-02, US-03
FR-06	Check-in	Move a booking to in use inside the check-in window and record the attendance entry	Booking number, current time	State and check-in time	Must	US-04
FR-07	Temporary leave and return	Record the start and end of a leave; a leave beyond the limit is judged a timeout	Booking number, action time	State and remaining return time	Must	US-05
FR-08	Session end and timeout release	Students end a session; the system judges missing check-ins and expired leaves and releases the seat	Booking number, current time	State and release result	Must	US-06, US-07
FR-09	Cancellation	Cancel a pending booking and release the seat	Booking number	State and release result	Should	US-08
FR-10	Resource closure	Close a period of a study room or seat, cancel affected bookings and send notices	Target, period, reason	Affected bookings and the result	Should	US-10
FR-11	Record query	Query bookings, attendance records and operation logs with filters and paging	Filters, paging	Record list	Should	US-11, US-12
FR-12	Rule configuration	Maintain rooms, seats, opening hours and rule parameters, with a version number	Resources, hours, parameters	Save result and version	Must	US-09
FR-13	Operation log	Record sign-ins, booking state changes, rule changes and resource closures	Operator, action, target, time	Log entries	Should	US-09, US-11
FR-14	In-system notices	Send closure reasons and timeout reminders to the affected students	Booking or resource events	In-system notice	Could	US-10
FR-15	Data export	Export bookings and attendance records that match the current query	Query conditions	CSV file	Could	US-11

4.2 Business rules and state model

The difficulty in a booking system is not the pages but the rules. Once the rules are written down, the code is largely a translation of them. The tables below list the rules, then the states of a booking and the transitions between them, and a diagram summarises the transitions. Values marked as provisional come from the Assignment 2 proposal and need confirmation by an administrator before they are treated as official rules.

Table 4-2 Business rules

ID	Rule	Value or decision	Source and status
BR-01	Maximum booking length	No more than 4 hours (maxDurationHours)	Provisional, pending administrator confirmation (TBD-02)
BR-02	Booking lead time	Not before today and no more than 7 days ahead (maxAdvanceDays)	Provisional, pending confirmation (TBD-02)
BR-03	The period must fall inside opening hours	Start and end must both be covered by opening hours and must not cross a closing time	Configured by the administrator (FR-12)
BR-04	Check-in window	From 10 minutes before the start to 15 minutes after it, including the later boundary; missing it means the booking is expired	Provisional, pending confirmation (TBD-02, TBD-04)
BR-05	Overlap decision	Periods are half-open intervals [start, end); two periods overlap exactly when start1 < end2 and start2 < end1, so adjacent periods do not overlap	Settled
BR-06	One valid booking at a time	At most one valid booking per seat and per student in overlapping periods; periods freed by cancellation, expiry or completion become bookable again	Settled
BR-07	Closed periods accept no bookings	Requests inside a closed period are rejected; existing affected bookings are cancelled after the administrator confirms	Settled
BR-08	Maximum temporary leave	No more than 30 minutes (tempLeaveLimitMin); beyond that the booking is expired	Provisional, pending confirmation (TBD-05)
BR-09	State idempotency	Repeated cancel, check-in, leave, return or end requests do not change recorded times and cause no extra side effects	Settled
BR-10	Concurrent writes	Validation and writing happen in one transaction; at most one of two competing requests succeeds and the others receive a retryable reason	Settled
BR-11	Rule and resource changes are recorded	Performed by administrators with operator, time and reason recorded, and a new rule version	Settled
BR-12	Time basis	Server time decides window and timeout questions; pages display Beijing time	Settled
BR-13	Record retention	Bookings, attendance and operation logs are kept during the project; the retention period and deletion mechanism must be confirmed before real deployment	Open (TBD-06)

A booking has six states. States are modelled as an enumeration rather than a boolean such as "valid or not", because expiry, cancellation and completion are all terminal but carry different meanings and responsibilities, and dispute handling needs to tell them apart.

Table 4-3 Booking states

State	Name	Meaning	Terminal	Allowed actions
PENDING_CHECKIN	Awaiting check-in	The booking exists and waits for check-in inside the window	No	Check in, cancel
IN_USE	In use	Checked in and occupying the seat	No	Leave, end session
ON_LEAVE	On leave	Temporarily away, waiting to return	No	Return, end session
COMPLETED	Completed	The student ended the session	Yes	View only
EXPIRED	Expired	No check-in inside the window, or the leave limit was exceeded	Yes	View only
CANCELLED	Cancelled	Cancelled by the student or by an administrator closing the resource	Yes	View only

Table 4-4 Booking state transitions

ID	From	Event	To	Condition and idempotency
TR-01	—	Booking submitted	Awaiting check-in	Rule and conflict checks pass; a booking number is returned
TR-02	Awaiting check-in	Check in	In use	Current time inside the check-in window; a repeated check-in keeps the first time
TR-03	Awaiting check-in	Check-in window closes	Expired	Triggered by the scheduled task or by a read or write; the first decision time is kept
TR-04	Awaiting check-in	Student cancels	Cancelled	Cancellation time recorded; repeated cancellations change nothing
TR-05	In use	Leave	On leave	Leave start recorded; whether a new request restarts the timer is open (TBD-05)
TR-06	On leave	Return	In use	Return within 30 minutes; the leave time counts towards the session
TR-07	On leave	Leave limit exceeded	Expired	No return inside the limit; the remaining period is released
TR-08	In use	Student ends session	Completed	End time recorded and the remaining period released
TR-09	In use	Booked end time reached	Completed	Decided by the scheduled task and recorded differently from an early end
TR-10	Awaiting check-in, in use or on leave	Administrator closes the resource	Cancelled	Operator, time and reason recorded; an in-system notice is sent
TR-11	Any terminal state	Repeated request	Unchanged	Handled idempotently with no extra side effects

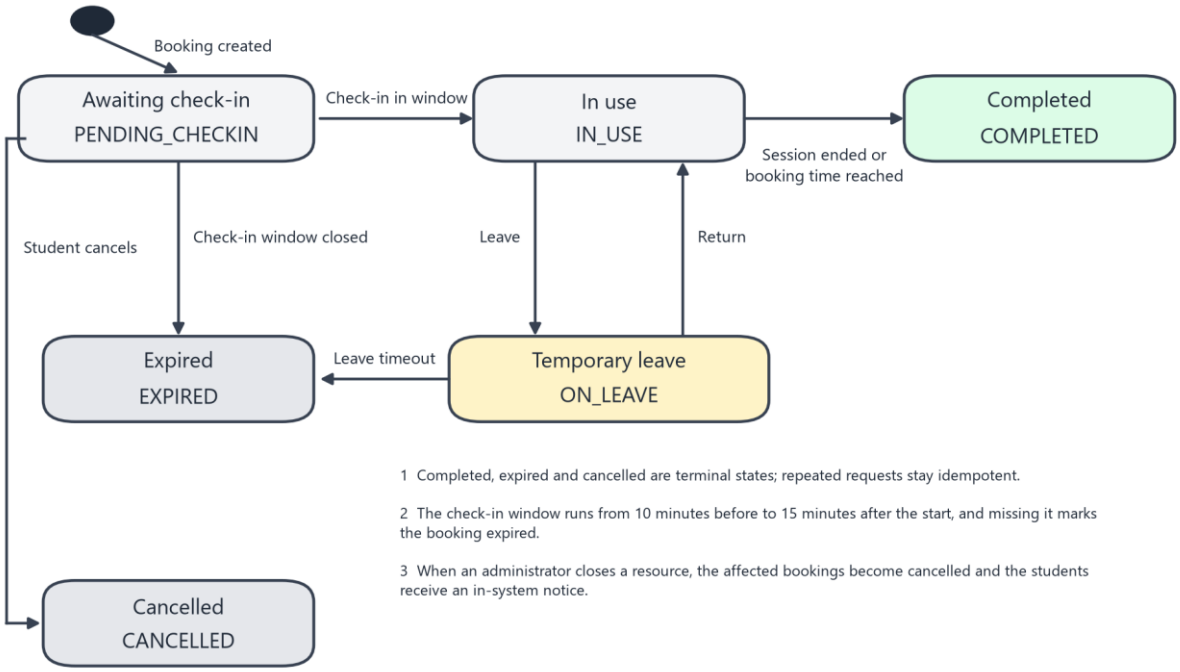
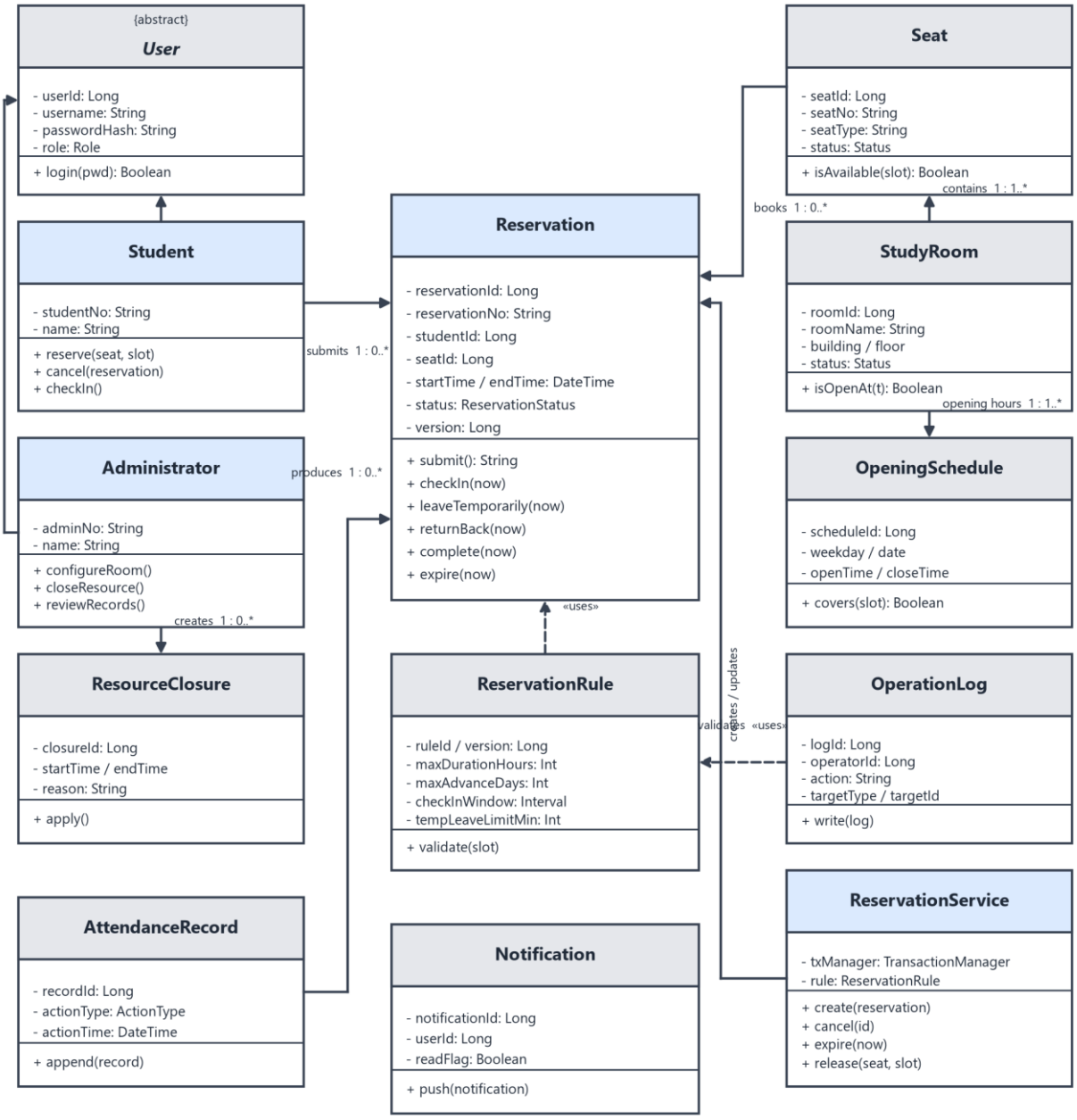


Figure 4-1 Booking state transitions

4.3 Class diagram

The class diagram fixes the domain vocabulary so that the same concept is not named differently on different pages. Three separations were made deliberately. First, resource classes and usage record classes are kept apart: study rooms, seats and opening hours describe what exists and when it is usable, while bookings and attendance records describe who used what and when. Second, parameters are concentrated in ReservationRule, so that pages and business code read the same values and a rule change does not require editing several decisions. Third, conflict checking and the transaction boundary live in ReservationService rather than being spread over page logic, because concurrency control can only be verified when it is handled in one place.



Solid lines are associations and dashed lines are dependencies; multiplicities are labelled.

ReservationService writes OperationLog entries and pushes Notification messages when a booking changes state.

ReservationService checks and writes seat and student time conflicts inside one transaction.

Figure 4-2 System class diagram

Table 4-5 Class descriptions

Class	Responsibility	Key attributes	Key operations
User (abstract)	Holds account and role data and verifies sign-in	userId, username, passwordHash, role	login(pwd)
Student	Student role, starts booking and attendance actions	studentNo, name	reserve, cancel, checkIn
Administrator	Administrator role, maintains resources and rules, handles exceptions	adminNo, name	configureRoom, closeResource, reviewRecords
StudyRoom	A study room that groups seats and opening hours	roomId, roomName, building/floor, status	isOpenAt(t)
Seat	A seat, deciding availability for a given period	seatId, seatNo, seatType, status	isAvailable(slot)
OpeningSchedule	Weekly opening hours and one-off adjustments	scheduleId, weekday/date, openTime/closeTime	covers(slot)
Reservation	The booking itself, holding the period and state and performing state changes	reservationNo, studentId, seatId, startTime/endTime, status, version	submit, checkIn, leaveTemporarily, returnBack, complete, expire
AttendanceRecord	Check-in and leave details forming the attendance timeline	recordId, actionType, actionTime	append(record)
ReservationRule	Rule parameters and the validation entry point	ruleId, version, maxDurationHours, maxAdvanceDays, checkInWindow, tempLeaveLimitMin	validate(slot)
ReservationService	Transaction boundary for booking use cases: create, cancel, expire and release	txManager, rule	create, cancel, expire, release
ResourceClosure	A closure record and the bookings it affects	closureId, startTime/endTime, reason	apply()
OperationLog	Audit record of key actions	logId, operatorId, action, targetType/targetId	write(log)
Notification	In-system notice explaining a reason and suggesting a replacement	notificationId, userId, readFlag	push(notification)

Table 4-6 Associations between classes

Association	Type	Multiplicity	Description
Student and User	Generalisation	—	A student is one kind of user and shares the account data and sign-in entry
Administrator and User	Generalisation	—	An administrator is another kind of user with additional rights on resources and rules
Student and Reservation	Association	1 : 0..*	One student may hold many bookings and each booking belongs to one student

Association	Type	Multiplicity	Description
Seat and Reservation	Association	1 : 0..*	A seat is booked many times over different periods but at most once per period
StudyRoom and Seat	Association	1 : 1..*	A study room contains several seats
StudyRoom and OpeningSchedule	Association	1 : 1..*	A study room has several opening hour records
Reservation and AttendanceRecord	Association	1 : 0..*	One booking produces several attendance entries forming the timeline
Reservation and ReservationRule	Dependency	—	Submission and state decisions validate against the current rule version
ReservationService and Reservation	Dependency	—	The service layer creates and updates bookings inside one transaction
Administrator and ResourceClosure	Association	1 : 0..*	Administrators initiate closures and state the reason
Administrator and ReservationRule	Association	1 : 0..*	Administrators maintain rule versions; the line is omitted from the figure to keep it readable

The trade-off around concurrency deserves a separate note. The intuitive approach is to check whether a seat is free before submitting on the page, but two requests can pass that check at the same moment and both write afterwards, producing a duplicate booking. The system therefore re-checks and writes inside a single transaction at submission time, and the database enforces a uniqueness constraint on valid bookings for the same seat and period; the version attribute of a reservation supports optimistic locking and retries. With that in place, even simultaneous requests leave at most one booking behind.

4.4 Data and consistency requirements

- **Uniqueness:** At most one valid booking exists per seat in overlapping periods, and likewise per student. A database unique index and checks inside the transaction together provide this; either alone is not enough.
- **Transaction boundary:** Creating, cancelling, checking in, leaving, returning, ending and expiring a booking all complete inside a transaction; a failed check rolls the whole change back so no partial write remains.
- **Concurrency control:** Lock the seat and period, or use a version number for optimistic locking. A conflict returns a retryable error and the page asks the student to choose another seat instead of failing silently.
- **Time representation:** Start and end times are stored in date-time columns rather than strings, and overlap queries follow the condition in BR-05.
- **Indexes:** A combined index on seat and time supports conflict checks, and a combined index on student and time supports personal record queries. Removing an invalid booking keeps an audit trace.

• **Expiry handling:** A scheduled task handles expiry periodically and read or write interfaces check again. Because both paths can fire, expiry has to be idempotent (BR-09) and record the decision time only once.

4.5 Interface requirements

Interfaces are the contract between front end and back end. The table lists the main interfaces; paths and methods may be refined during implementation, but the meaning of inputs and outputs and the error codes must stay as specified here, otherwise the pages cannot report failures reliably. Every interface returns { code, message, data }, and list interfaces also return total, page and pageSize.

Table 4-7 Main interfaces

ID	Method and path	Purpose	Main input	Main output	Requirements
API-01	POST /api/auth/login	Sign in and return the role	username, password	token, role, displayName	FR-01
API-02	POST /api/auth/password	Change password	Old and new password	Result of the operation	FR-02
API-03	GET /api/seats/available	Search free seats	date, start, end, type	Seat list with states	FR-03
API-04	POST /api/reservations	Submit a booking	seatId, start, end	reservationNo or an error code	FR-04, FR-05
API-05	POST /api/reservations/{no}/checkin	Check in	reservationNo	State, check-in time	FR-06
API-06	POST /api/reservations/{no}/leave	Take a temporary leave	reservationNo	State, remaining return time	FR-07
API-07	POST /api/reservations/{no}/return	Return from leave	reservationNo	State	FR-07
API-08	POST /api/reservations/{no}/complete	End the session	reservationNo	State, session length	FR-08
API-09	POST /api/reservations/{no}/cancel	Cancel the booking	reservationNo	State	FR-09
API-10	GET /api/reservations/mine	Personal bookings and attendance	Paging, state filter	Booking and attendance list	FR-11
API-11	PUT /api/admin/rules	Save rule parameters	Parameter values	New version number	FR-12
API-12	POST /api/admin/closures	Close a resource	Target, period, reason	Affected bookings and result	FR-10
API-13	GET /api/admin/records	Query records	Filters, paging	Bookings, attendance, logs	FR-11, FR-13
API-14	GET /api/admin/records/export	Export records	Current filters	CSV file	FR-15

Table 4-8 Error codes and page handling

Error code	HTTP	Meaning	Page handling
SEAT_CONFLICT	409	The seat was taken at the moment of submission	Explain the reason, keep the filters and let the student pick another seat
STUDENT_CONFLICT	409	The student already holds a valid booking in that period	Show the conflict and the existing booking number
OUT_OF_OPEN_HOURS	422	The period is outside opening hours	Show the bookable periods
RULE_VIOLATION	422	Length or lead time exceeds the configured limit	Show the limit and the accepted values
CHECKIN_WINDOW_CLOSED	409	Check-in is outside the window and the booking is expired	Show the window and the current state
STATE_INVALID	409	The current state does not allow the action	Show the current state and the actions that are possible
UNAUTHORIZED	401	Not signed in or the session expired	Go to the sign-in page and return afterwards
FORBIDDEN	403	No permission for the resource or interface	Report the lack of permission without showing other users' data
INTERNAL_ERROR	500	Server error	Ask the user to retry and log the failure for diagnosis

4.6 Non-functional requirements

Non-functional requirements decide whether the system is pleasant to use and whether its results can be trusted. The performance target follows the Assignment 2 proposal and is used to judge whether the goal has been met; the test report has to record hardware, data volume, network conditions and duration rather than reporting a single number.

Table 4-9 Non-functional requirements

Area	ID	Requirement	Verification
Usability	NFR-U1	A student reaches a completed booking within three pages and the primary action is visible without scrolling	Usability test record
Usability	NFR-U2	States are expressed with text as well as colour, so colour alone never carries the meaning	Interface review (Table 5-4)
Usability	NFR-U3	Failure messages state the reason and the next action, never a bare error	Acceptance cases (Section 6.3)
Performance	NFR-P1	At 100 concurrent virtual users, 95 percent of search and booking operations respond within 2 seconds	Load test report (AC-15)
Performance	NFR-P2	In the concurrency test, the number of duplicate successes for one seat and period is zero	Conflict test report (AC-05)

Area	ID	Requirement	Verification
Performance	NFR-P3	The first screen loads within 3 seconds on the campus network (reference target, not a hard acceptance condition)	Manual timing record
Security	NFR-S1	Passwords are stored with a secure hash such as bcrypt and never appear in logs or responses	Code and log review
Security	NFR-S2	Every interface checks the role and ownership on the server, so a student can only reach personal bookings	Interface test (AC-14)
Security	NFR-S3	Inputs are validated by type and range and database queries are parameterised to avoid injection	Code review
Reliability	NFR-R1	A seat is never occupied without a valid booking, and cancelled, expired or completed periods can be booked again	Consistency test (AC-06, AC-13)
Reliability	NFR-R2	After an interruption the scheduled task catches up on expired bookings so resources are not blocked	Failure drill record
Maintainability	NFR-M1	Rule parameters are held in configuration, so changing them needs no code change	Configuration review
Maintainability	NFR-M2	Key actions leave logs that identify the booking number and the operator	Log review
Compatibility	NFR-C1	The last two versions of the common desktop and mobile browsers are supported	Compatibility review
Portability	NFR-T1	The system can be deployed on a course lab machine or a campus server with scripted dependency installation	Deployment document and drill

5 Interface prototype

5.1 Design principles and information architecture

The interface design follows three principles. First, state is visible: which state a booking is in and what can be done next are always on screen, so users do not have to remember the rules. Second, one confirmation per consequential action: submitting a booking, cancelling and closing a resource all ask for confirmation and show the scope of the change beforehand. Third, failures are recoverable: when something goes wrong the page keeps the choices already made and shows the next action instead of sending the user back to the start.

The information architecture separates the student side from the administrator side, and each has one main path. For students it is sign-in, seat search, booking confirmation, and my bookings with check-in. For administrators it is sign-in, resources and rules, record query, and exception handling. Both sides use the same state names as the state table in Section 4.2, so the page never says “overdue” while the interface says “expired”.

Table 5-1 Pages and information architecture

Side	Page	Purpose	Key actions
Student	Sign in	Confirm identity and role	Enter account and password
Student	Seat search	Find a free seat that matches the filters	Filter, search, select a seat
Student	Booking confirmation	Check the details and submit	Confirm booking, go back to search
Student	My bookings and attendance	See the booking state and act on it	Check in, leave, return, end, cancel
Administrator	Rule configuration	Maintain resources, opening hours and rule parameters	Save rules, restore defaults
Administrator	Resource closure	Stop a period and handle the affected bookings	Choose target and period, confirm closure
Administrator	Usage records and logs	Investigate usage and disputes	Query, inspect details, export

5.2 Student interface

The visual direction for the student side was settled in Assignment 2: a light palette with blue as the single accent colour, and states marked by both text and background tint, which is what NFR-U2 requires. The concept image below keeps that direction, while the wireframes add the confirmation and attendance screens that it did not cover.

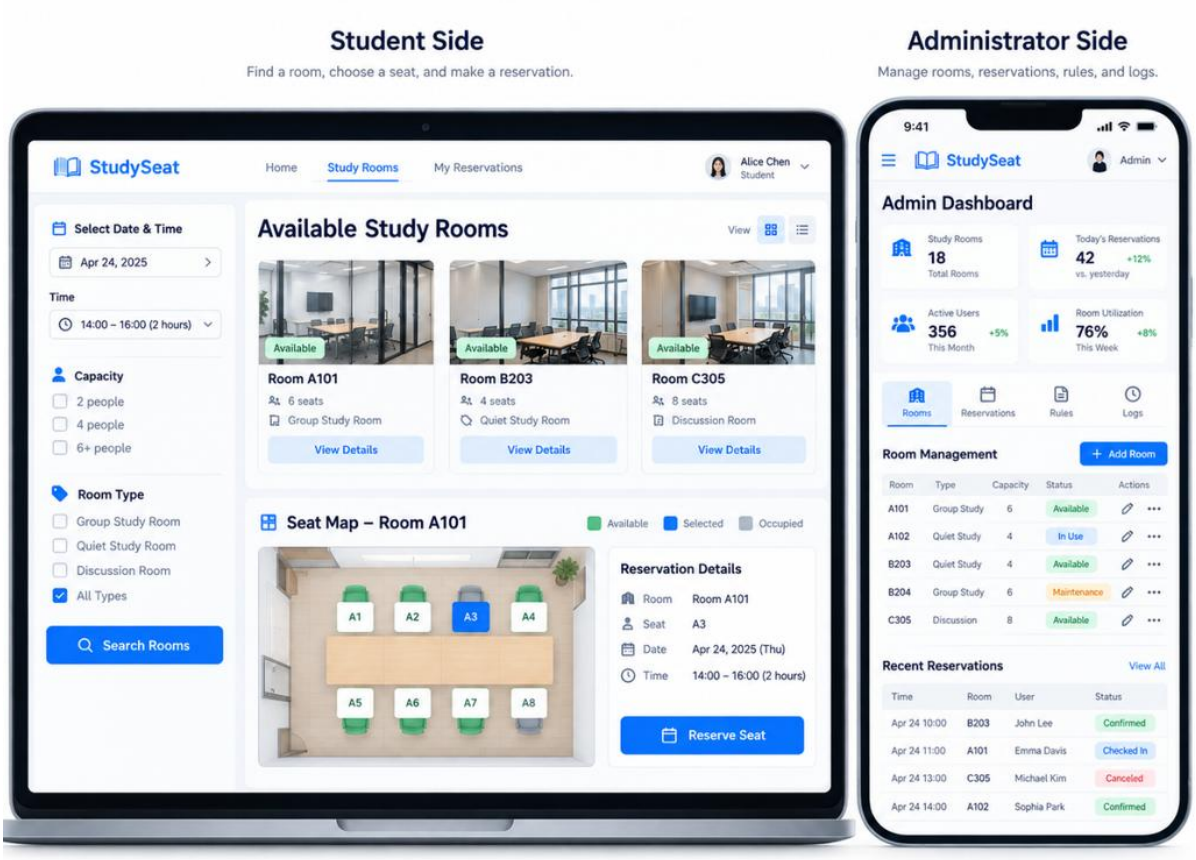


Figure 5-1 Interface concept for the student and administrator sides (visual direction)

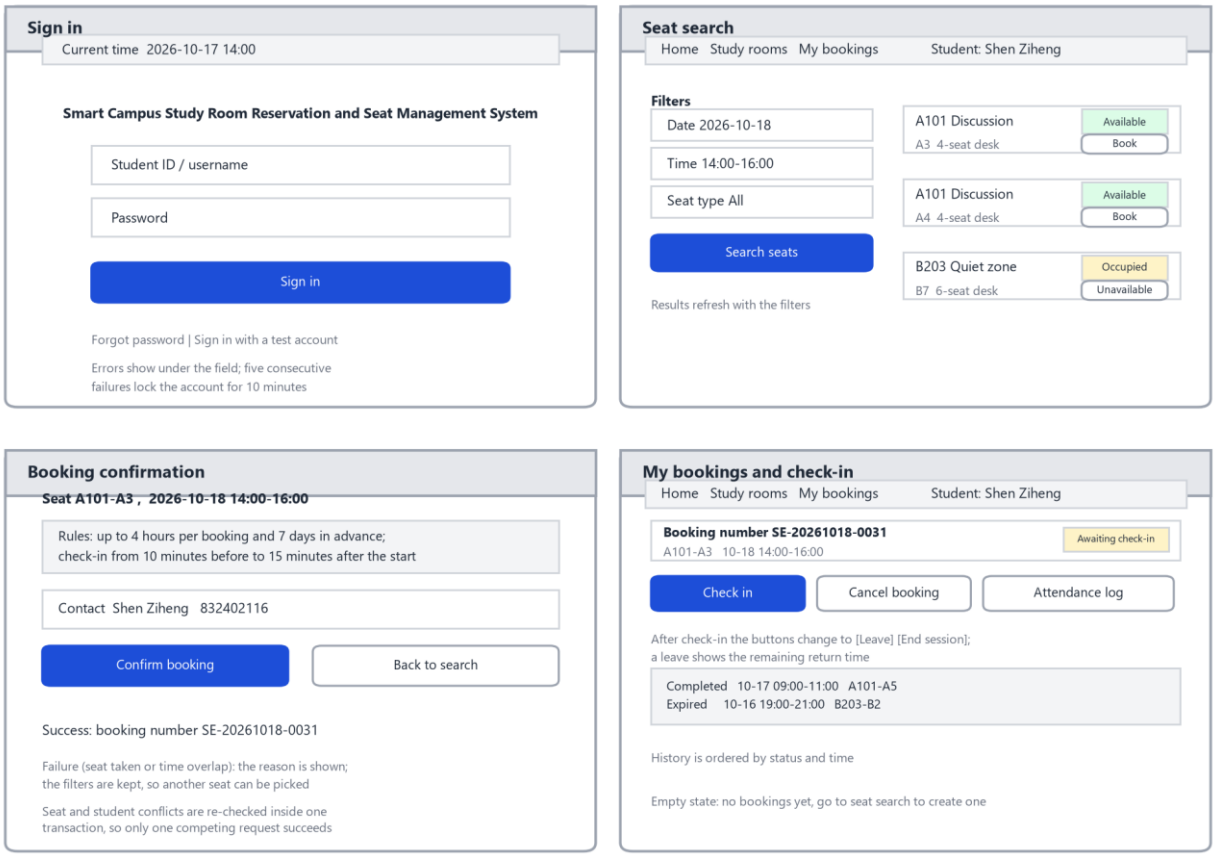


Figure 5-2 Student wireframes for the key screens

Table 5-2 Student screens and failure handling

Screen	Main elements	Interaction and state changes	Failure handling
Sign in	Student ID field, password field, sign-in button, password help	The button is disabled while the request runs; success returns to seat search	The reason appears under the field; five consecutive failures lock the account for 10 minutes
Seat search	Navigation, filters for date, period, capacity and seat type, search button, seat cards	Changing a filter refreshes the list; the book button of an unavailable seat is disabled	An empty state suggests nearby periods; a failed request offers retry and keeps the filters
Booking confirmation	Seat and period summary, rule reminders, confirm and back buttons	On success the booking number and check-in window are shown	A conflict or a rule violation is explained and the chosen filters are kept for another seat
My bookings and attendance	Booking cards, state labels, check-in, leave, return and end buttons, history	The available buttons follow the state; a leave shows the remaining return time	Actions that the state does not allow are disabled with a reason; no records shows an empty state linking to search

5.3 Administrator interface

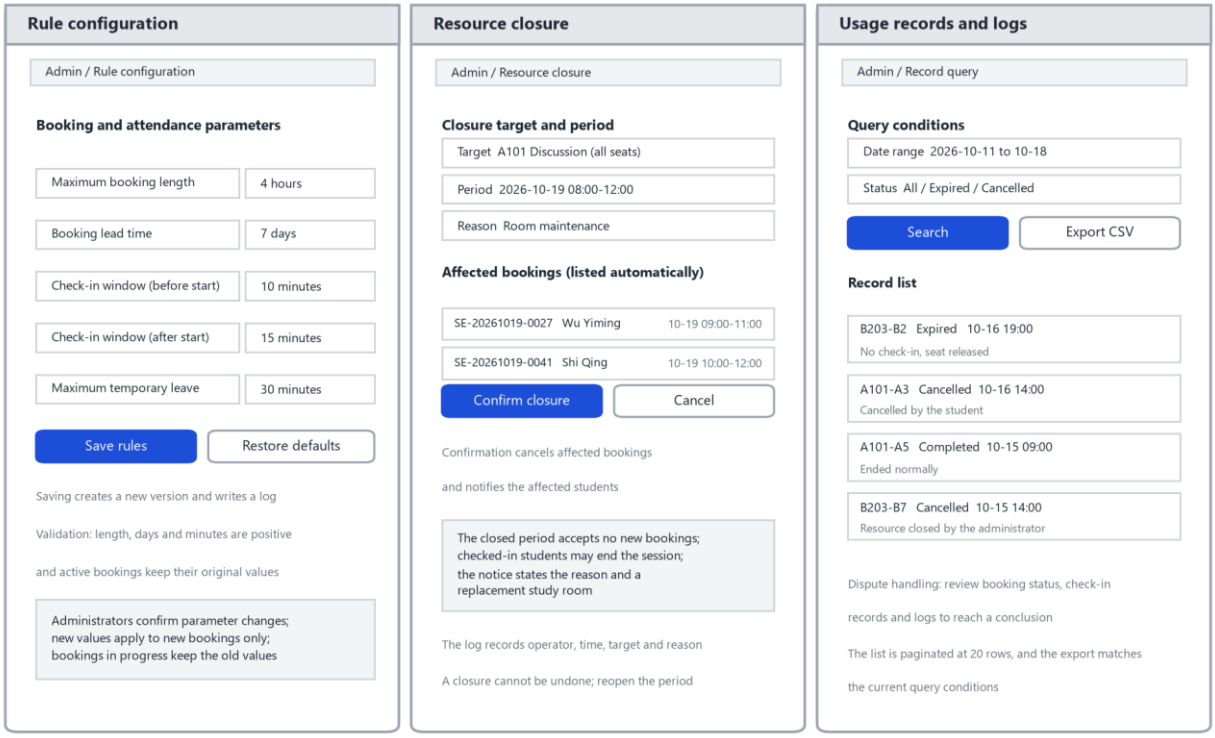


Figure 5-3 Administrator wireframes for the key screens

Table 5-3 Administrator screens and failure handling

Screen	Main elements	Interaction and state changes	Failure handling
Rule configuration	Parameter fields, save rules, restore defaults	After saving, the new version number and effective scope are shown	An invalid value is rejected and its field is highlighted; changes apply to new bookings only
Resource closure	Target and period selection, affected booking list, reason, confirm closure	Selecting a period lists the affected bookings and the count is shown before confirmation	A booking already in use asks for on-site handling; the period must be reopened afterwards
Usage records and logs	Filter fields, record list, search and export	The list is paginated and the export matches the current filters	Missing records prompt an explanation of the retention period and a note to contact the maintainer

5.4 Screen sequence for the main flow

The figure below connects the student screens in time order and shows where the exception paths lead. The exceptions deliberately return to a state where the student can choose again instead of dropping back to sign-in: a taken seat or an overlapping period can be resolved immediately, and asking the user to re-enter everything would shift the system's responsibility onto the person using it.

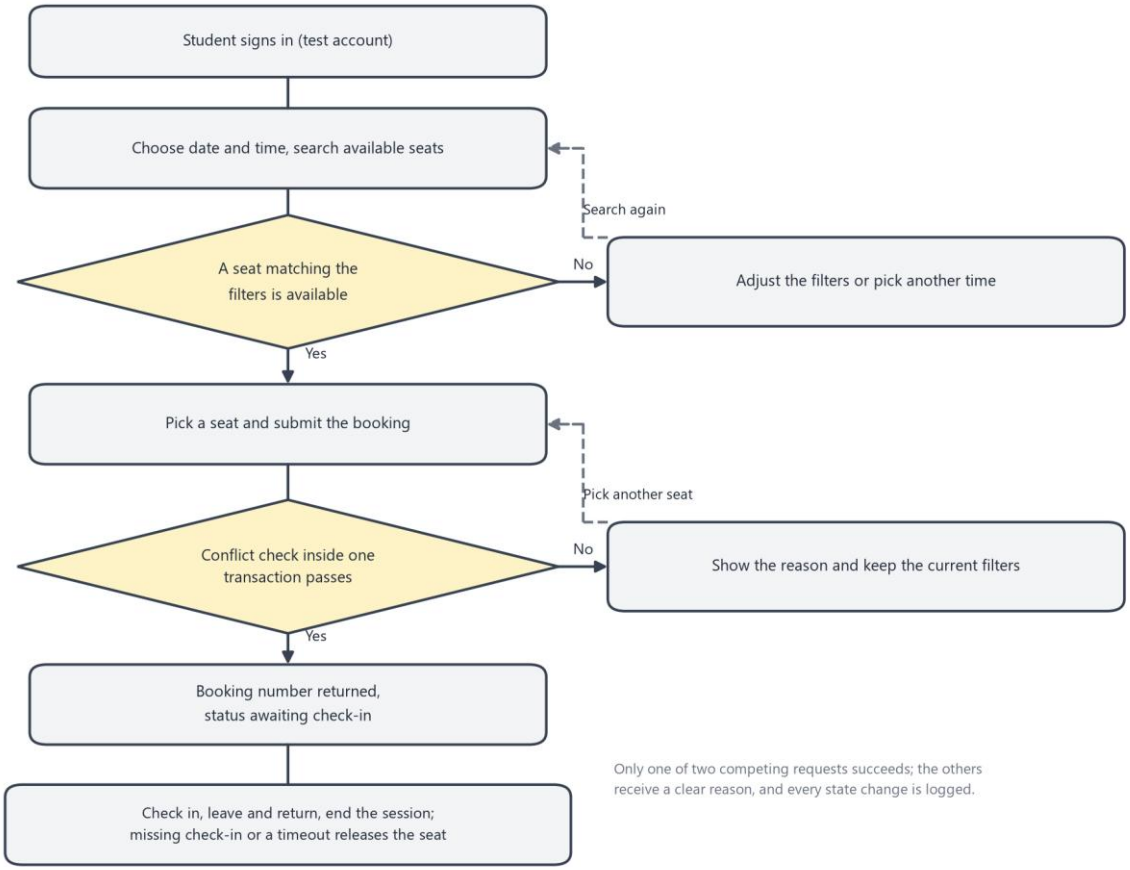


Figure 5-4 Main booking flow and exception paths

5.5 Interface consistency and usability checks

The checks below are run once the interfaces are built. There are only a few, but each one corresponds to a concrete failure: inconsistent state names make users suspect the system is broken, and expressing state by colour alone leaves users with colour vision deficiency unable to tell what is happening.

Table 5-4 Interface consistency and usability checklist

Check	Requirement	Method
Consistent terms	State names and button labels match the state table in Section 4.2	Page by page against the state table
Clear primary action	Each page has exactly one prominent primary action	Page walkthrough
Action feedback	Submitting, cancelling and checking in all give success or failure feedback	Manual operation record
Recognisable states	State is shown as text as well as colour	Interface review (NFR-U2)
Small screens	On a phone width the key buttons are not obscured and forms remain usable	Mobile walkthrough

Check	Requirement	Method
Recoverable errors	After a failure the chosen filters are kept and a next action is offered	Force a conflicting submission
Acceptance alignment	Interface acceptance criteria match Section 6.3 one by one	Cross-check against Section 6.3

6 Functional description and acceptance criteria

This chapter turns the requirements of Chapter 4 into something that can be executed and checked. The first part states what each function receives, how it processes it, what it returns and what happens on failure; the second part gives acceptance criteria and cases that can be followed directly, so that acceptance does not require re-interpreting the intent of a requirement.

6.1 Detailed functional description

6.1.1 Sign-in and permissions (FR-01, FR-02)

The sign-in interface receives an account and password, compares the password against the stored hash, issues a session token on success and returns the role. On failure it returns one common message without distinguishing an unknown account from a wrong password, so that error messages cannot be used to probe accounts. The token accompanies every later request, and each interface checks the role and the ownership of the data on entry: students read only their own bookings and attendance, and administrator interfaces accept only the administrator role. Five consecutive failures lock the account for 10 minutes, with the remaining time shown on the page. Changing a password requires the old one and a new one that meets the length requirement, after which the previous session becomes invalid.

6.1.2 Seat search and resource states (FR-03, FR-12)

The search interface receives a date, a period and optional filters. The server first checks whether the requested period falls inside opening hours, then selects the seats with no valid booking overlapping that period according to BR-05, and returns them with seat type and study room. Seat availability is computed for a period rather than stored as a single “free” flag, because a seat occupied in the morning can still be booked in the afternoon. Searching writes nothing; if the period is outside opening hours the interface returns an error code and the page shows the periods that are bookable. On the

administrator side, the resource interface receives rooms, seats, opening hours and rule parameters, validates the value ranges before saving and issues a new rule version on success.

6.1.3 Booking submission and conflict check (FR-04, FR-05)

Submission is the step that needs the most care. The interface receives a seat, a start and an end time, and the server applies four checks in order: whether the period is inside opening hours (BR-03), whether the length exceeds the maximum (BR-01), whether the booking is too far ahead (BR-02), and whether the student already holds a valid booking in that period (BR-06). If they pass, the server locks the seat and period inside one transaction, confirms once more that there is no conflict, then writes the booking, generates the booking number and returns the start and end of the check-in window. Any failure rolls the whole change back and returns the matching error code with a readable reason while the page keeps the filters so that another seat can be chosen. Repeating the same submission is handled idempotently and does not create a second booking.

6.1.4 Check-in, temporary leave and return (FR-06, FR-07)

The check-in interface decides from server time whether the window is open (BR-04). Inside the window, the state moves from pending to in use, the check-in time and method are recorded and an attendance entry is added. Before the window opens the interface reports that check-in is not available yet together with the opening time; after it closes the booking has already been marked expired by the expiry handling, so the interface returns the current state rather than a bare error and the page reports that the seat has been released. Leave and return record the start and end of the leave and set the state back to in use, with the leave time counting towards the session; while away, the page shows the remaining return time and the expiry handling marks the booking expired once the limit passes. These actions are idempotent: a repeated check-in does not overwrite the first check-in time and a repeated leave does not add a second attendance entry.

6.1.5 Session end and timeout release (FR-08, FR-09)

Ending a session marks a booking in use or on leave as completed, records the end time and releases the part of the period after the current moment, with the session length displayed. Cancellation is accepted only in the pending state; after confirmation the booking becomes cancelled and the seat is released, while a booking already checked in or

completed cannot be cancelled and the page points to ending the session instead. Timeout release is performed both by the scheduled task, which scans pending and on-leave bookings, and by validation during read or write operations when a student or administrator views records. Both paths must be idempotent so that a booking records its decision time once and repeated releases cannot make the state jump.

6.1.6 Administrator configuration, closures and record queries (FR-10 to FR-15)

Saving rule configuration creates a new version that applies only to bookings submitted afterwards, while bookings already under way keep the previous parameters; this is why adjusting parameters cannot change the rules of a student who has already confirmed a booking. The closure interface receives the target, period and reason, returns the affected bookings for confirmation, and on confirmation cancels them, writes an audit record and generates in-system notices, while no new booking is accepted inside the closed period. The record interface filters by date, seat, student number and state, returns bookings, attendance and operation logs with paging, and exports a CSV file matching the current filters; the exported column order is fixed so that the file can be checked in a spreadsheet tool.

6.2 Input and output formats

Time and number formats are defined by the server; pages display them but never parse them again. All time decisions use server time (BR-12), so that a modified client clock cannot bypass the check-in window.

Table 6-1 Time, number and file formats

Item	Format	Example	Note
Date	yyyy-MM-dd	2026-10-18	Identical on page and server, no time zone suffix
Time	HH:mm	14:00	24-hour clock
Date and time	yyyy-MM-dd HH:mm:ss	2026-10-18 14:00:00	Interface and database storage format
Period input	Start and end submitted separately	start=2026-10-18 14:00, end=2026-10-18 16:00	Represents the half-open interval [start, end)
Booking number	SE-yyyyMMdd-sequence	SE-20261018-0031	Unique; the page offers a copy action
Export file	CSV, UTF-8 with BOM	reservations_20261018.csv	Fixed column order so that spreadsheet tools open it directly
Interface response	JSON	\{ code, message, data \}	Lists also return total, page and pageSize

Table 6-2 Validation rules for the main input fields

Field	Type	Required	Constraint	Failure message
username	String	Yes	4 to 20 characters, letters or digits	Report the expected format
password	String	Yes	8 to 32 characters	Report the length requirement without echoing input
date	Date	Yes	Not before today and not more than 7 days ahead (BR-02)	Show the accepted date range
start / end	Date-time	Yes	End after start; length at most 4 hours (BR-01); inside opening hours (BR-03)	Report each rule that is not met
seatId	Integer	Yes	The seat exists and is enabled	Report that the seat is unavailable
reservationNo	String	Yes	Well formed and owned by the signed-in student	Handled as FORBIDDEN when not owned
reason	String	Yes	Required for a closure, 2 to 200 characters	Ask for the missing reason

6.3 Interface acceptance criteria

Interface acceptance is about whether users can finish the task themselves. Each criterion corresponds to an action that can be demonstrated on the spot; the table is worked through item by item during acceptance and anything that fails goes back to Chapter 5.

Table 6-3 Interface acceptance criteria

Screen	Check	Pass condition
Sign in	Wrong input	The message appears under the field and the page does not navigate; after five failures the remaining lock time is shown
Seat search	Changing filters	The list updates with the filters within 2 seconds and state labels match the table in Section 4.2
Seat search	No results	An empty state suggests nearby periods and the chosen filters are kept
Booking confirmation	Conflicting submission	The page reports that the seat was taken, keeps the filters and allows another seat to be selected
Booking confirmation	Rule violation	The length, lead time and opening hour limits are reported individually
My bookings and attendance	State buttons	The buttons match the current state, and unavailable actions are disabled with a reason
My bookings and attendance	Leave timer	The remaining return time is shown and the state returns to in use after returning
Rule configuration	Invalid parameter	Saving is refused, the field is highlighted and other fields keep their values

Screen	Check	Pass condition
Resource closure	Scope of impact	The number and periods of the affected bookings are shown before confirmation and the notice is visible afterwards
Usage records and logs	Export	The exported file matches the current filters and keeps the fixed column order

6.4 Functional acceptance criteria

The acceptance cases follow a minimal reproducible form: preconditions are explicit, steps can be executed as written and expected results can be judged yes or no. For cases involving concurrency, the reason returned to the unsuccessful request also has to be recorded, otherwise the case shows only that nothing went wrong by luck.

Table 6-4 Functional acceptance cases

ID	Case	Preconditions	Steps	Expected result	Req.
AC-01	Sign-in and role	Test accounts exist	Sign in with a student account, then call an administrator interface	Sign-in succeeds with the student role; the administrator interface returns 403	FR-01, NFR-S2
AC-02	Seat search	Opening hours and seats are configured	Search by date and period	Seats that are free are returned and occupied seats carry their state	FR-03
AC-03	Successful booking	The seat is free for the period	Submit a booking and inspect the result	A booking number is returned, the state is awaiting check-in and a log entry is added	FR-04, FR-13
AC-04	Length and lead time	Rules are 4 hours and 7 days	Submit a 5-hour booking, then one 8 days ahead	Both are rejected with the limit that was exceeded	FR-04, BR-01, BR-02
AC-05	Concurrent booking conflict	One seat and one period	Send 100 concurrent requests for the same seat and period	Exactly one succeeds, no duplicates exist and the others report a seat conflict	FR-05, NFR-P2, BR-10
AC-06	Release after a missed check-in	A pending booking exists	Trigger expiry handling after the window closes	The state becomes expired and the seat can be booked again for the remaining period	FR-08, BR-04
AC-07	Check-in window boundaries	A pending booking exists	Check in 10 minutes before the start and 15 minutes after it	The in-window action succeeds; the later action reports a closed window and the expired state	FR-06, BR-04
AC-08	Student period conflict	The student holds a valid booking	Submit a booking for another seat in an overlapping period	The booking is rejected and the existing booking number is shown	FR-05, BR-06

ID	Case	Preconditions	Steps	Expected result	Req.
AC-09	Adjacent periods allowed	The student booked 14:00 to 16:00	Submit a booking from 16:00 to 18:00	The booking succeeds because adjacent periods do not overlap	BR-05
AC-10	Leave and leave timeout	A booking is in use	Return from a leave within 30 minutes, then take a leave exceeding 30 minutes	The first returns to in use; the second becomes expired and releases the seat	FR-07, BR-08
AC-11	Resource closure handling	Affected bookings exist	An administrator closes the period and confirms	Affected bookings become cancelled, an audit record is written and notices are sent	FR-10, BR-13
AC-12	Terminal state idempotency	A booking is terminal	Repeat the cancel or end request	State and times do not change and no extra side effects occur	BR-09
AC-13	Seat released after cancellation	A pending booking exists	Cancel it, then book the same period as another student	The other student can book and the seat state is correct	FR-09, BR-06
AC-14	Permission isolation	Two student accounts exist	Student A requests student B's booking number	The response reports no permission or no such record and shows no data	NFR-S2
AC-15	Performance and stability	100 concurrent virtual users	Run a mixed search and booking load	95 percent of main operations respond within 2 seconds and the error rate is recorded in the report	NFR-P1

6.5 Requirements traceability

The traceability matrix confirms that no requirement is missing and that no acceptance case is homeless. When a requirement has a scenario in Chapter 3, an identifier in Chapter 4 and a case in Section 6.4, a reviewer can see that it was not forgotten somewhere in the document.

Table 6-5 Traceability of requirements, scenarios and acceptance cases

Requirement	Name	Scenarios	Acceptance cases
FR-01	Sign-in and role detection	US-01	AC-01
FR-02	Account and password management	US-12	AC-01
FR-03	Seat search	US-01	AC-02
FR-04	Booking submission	US-02, US-03	AC-03, AC-04
FR-05	Conflict check	US-02, US-03	AC-05, AC-08, AC-09
FR-06	Check-in	US-04	AC-07
FR-07	Temporary leave and return	US-05	AC-10
FR-08	Session end and timeout release	US-06, US-07	AC-06, AC-12

Requirement	Name	Scenarios	Acceptance cases
FR-09	Cancellation	US-08	AC-13
FR-10	Resource closure	US-10	AC-11
FR-11	Record query	US-11, US-12	AC-11, AC-14
FR-12	Rule configuration	US-09	AC-04
FR-13	Operation log	US-09, US-11	AC-03, AC-11
FR-14	In-system notices	US-10	AC-11
FR-15	Data export	US-11	AC-11

7 Open items and change management

7.1 Open items

The items below affect the requirements directly. Until they are confirmed the current assumption is implemented, and the marker is kept in the document and the blog so that an assumption is never quietly turned into a conclusion.

Table 7-1 Open items

ID	Item	Current assumption	Impact	Owner	Planned
TBD-01	Student interviews, at least 10	Finding and occupying seats is a common problem	Affects requirement priority and any narrowing of scope	Students on campus	Week 3
TBD-02	Administrator interview	Opening hours and rule parameters can be configured by the administrator	Affects the values of BR-01 to BR-04	Study room administrator	Week 3
TBD-03	Survey of existing platforms	No seat-level booking is available today	Affects the project positioning and may narrow the scope	Course team, administrator	Week 3
TBD-04	Check-in method	Check-in is a page action	Affects FR-06 and any anti-proxy measures	Study room administrator	Week 4
TBD-05	Whether a new leave restarts the timer	The timer is not restarted	Affects the exact definition of BR-08	Study room administrator	Week 4
TBD-06	Record retention and deletion	All records are kept during the project	Affects the privacy statement and data management	Administrator, course team	Week 5
TBD-07	Single sign-on and access gate integration	Version 1 uses test accounts	Affects the sign-in module and the conditions for a pilot	Campus IT office	Before a pilot

7.2 Change management

Requirement changes are inevitable, especially after the interviews. We handle them as follows: a change is first recorded in the team change log with the requester, the reason, the affected requirement identifiers and the expected time; the team leader and the requirements owner assess the impact and decide whether it enters the current scope; once approved, the version number of this document is updated together with the blog and the presentation so that the three materials state the same thing.

Rule parameter changes are handled separately: every save creates a new version that applies only to new bookings, while bookings already under way follow the previous parameters (BR-11). Adjusting a parameter therefore cannot confront a student who is

already out on campus with a new decision rule, and a dispute can be reconstructed against the rules that were in force at the time.

7.3 Risks and responses

Table 7-2 Main risks and responses

Risk	Possible symptom	Response
Requirements	An existing platform already covers the main needs and the project loses its point	Complete the platform survey first (TBD-03) and narrow the scope to what is missing, as described in Section 2.1.3
Rules	The check-in window or leave limit does not match how the room is actually run	Have an administrator confirm the values and store them in configuration (TBD-02, TBD-05); parameter changes need no code change
Technical	Concurrent submissions create duplicate bookings, or cancellation and expiry fail to release a seat	Checks inside a transaction plus a uniqueness constraint (BR-10), verified specifically by AC-05, AC-06 and AC-13
Schedule	Not enough time to finish every function	Follow the priority order in Section 2.5, secure the must-have items and explain the state of the rest during the demonstration
Data	Using real data would raise privacy concerns	Use simulated data and test accounts throughout; confirm the retention period and deletion mechanism before real deployment (TBD-06)
Coordination	The document, the presentation and the system disagree, which shows up during review	Treat this document as the single source for rule definitions; update the blog and presentation with every change and cross-check them

Appendix A Interview guide

The guide is ordered so that we first learn what people do today and only then test our assumptions. When recording answers, we keep actual experience separate from impressions, and for questions about frequency we ask for an approximate number of times rather than whether it “happens a lot”.

A.1 Questions for students

- When and where do you usually study, and roughly how many times a week is that?
- How do you tell whether a seat is free? How often in the past month did you arrive and find none?
- What do you do when there is no seat available, for example change floor or room?
- Have you seen a seat kept by belongings with nobody sitting there? What did you do?
- Is booking up to seven days ahead enough for you, or would you prefer longer?
- How long do you usually study in one session, and does a four-hour limit fit your habits?
- If you cannot attend, do you cancel beforehand, and in which situations do you forget?
- Would a check-in window from 10 minutes before to 15 minutes after the start work for you, and why?
- If you step away during a session, how long do you usually need, and is 30 minutes enough?
- Where would you look for your booking status, and what would you want the system to tell you when a booking fails?

A.2 Questions for administrators

- How are opening hours decided, and how are holidays and examination weeks handled?
- How far in advance are temporary closures known, and how are students informed today?
- How are long occupations and belongings without a person handled? Is there a written rule?
- Are temporary leaves allowed, for how long, and what happens when someone stays away longer?
- Should one student be limited in how many bookings they hold at once, and what limit is sensible?
- When a dispute arises, what information would you need from the system to judge it?
- How often do rooms and seats change, and would you rather maintain them yourself or through a technical person?
- Could single sign-on or access gates be integrated later, and how long should records be kept?

Appendix B Deliverables and links

This appendix lists where the deliverables of this submission are published. The document and the presentation are uploaded to GitCode, and the same links appear in the body of

the team blog post so that the document, the presentation and the blog never contradict each other.

Table B-1 Deliverables and links

Deliverable	Description	Link
Team CSDN home page	Entry point for the team's course work	https://blog.csdn.net/2501_91679864
Assignment 2 blog post	Source of the topic argument and of version V1.0 of this document	https://blog.csdn.net/2501_91679864/article/details/167582725
This specification	Word and PDF versions, uploaded to GitCode with this submission	The link is given in the team blog post
Requirements presentation	Used for the in-class presentation, uploaded to GitCode	The link is given in the team blog post
Course community and assignment page	Where the assignment is published and submitted	https://bbs.csdn.net/forums/2601_MU_SE_FZU